Chapter 4
Applied Search

Asst.Prof.Rapeeporn Chamchong, PhD.

Department of Computer Science, Faculty of Informatics, MSU

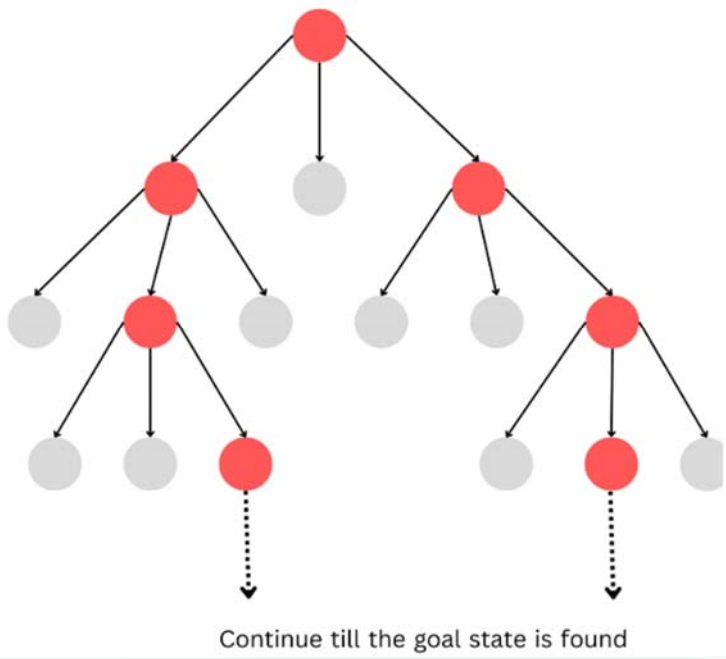
BEAM Search

- Modified version of the Greedy best-first search algorithm (Heuristic search strategy) using GBFS.
- Select nodes based on conditional probability.
- Each beam search iteration includes multiple pathways, ordered by path length, to identify the shortest or cheapest path.
- It evaluates and expands a set number, **W**, of the best nodes at each level, based on their heuristic values.

Characteristics of Beam Search

- **Width of the Beam (W)**: This parameter defines the number of nodes considered at each level. The beam width W directly influences the number of nodes evaluated and, hence, the breadth of the search.
- **Branching Factor (B)**: If B is the branching factor, the algorithm evaluates $W \times BW \times B$ nodes at every depth but selects only W for further expansion.
- **Completeness and Optimality**: Beam search's limited width can prevent it from finding the best solution by discarding potentially optimal paths.
- **Memory Efficiency**: The beam width bounds the memory required for the search, making beam search suitable for resource-constrained environments.

Working



- The search tree generated using the Beam search algorithm, assuming $W = 2$ and $B = 3$ is given in the example.
- Here, colored nodes are selected based on their heuristic values for further expansion.

Source: https://www.askpython.com/python/examples/beam-search-algorithm

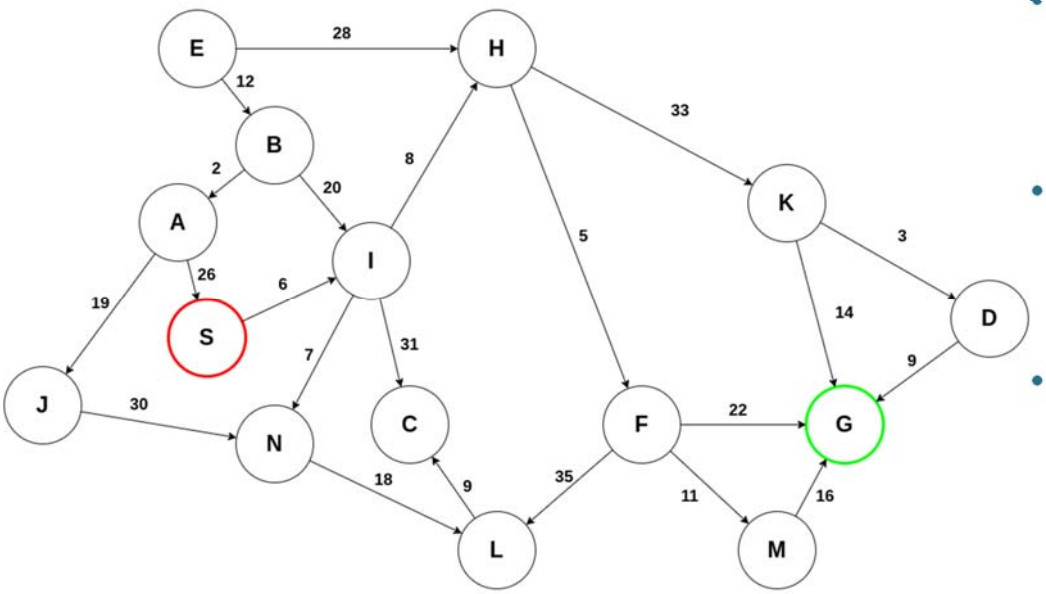
How does Beam search work?

- **Initialization**: Start with the root node (***S***) and generate its successors.
- **Node Expansion**: From the current nodes, generate successors and apply the heuristic function to evaluate them.
- **Selection**: Select the top ***W*** nodes according to the heuristic values. These selected nodes form the next level to explore.
- **Iteration**: Repeat the process of expansion and selection for the new level of nodes until the goal is reached or a certain condition is met (like a maximum number of levels).
- **Termination**: The search stops when the goal is found or when no more nodes are available to expand.

Beam Algorithm

- Use a **frontier list** and a **visited list** for the exploration of nodes.
  - Initialize a search tree with the root node (***S***). This is level zero in the tree.
  - Add ***S*** to the visited list and evaluate all nodes adjacent to ***S***.
  - Select the top ***W*** nodes & add them as child nodes to ***S*** in the tree, ignoring all others.
  - Add the selected nodes to the **frontier list** for further exploration.
  - Remove all nodes in the **frontier list**. Once a node is removed from the **frontier list** for exploration, add it to the **visited list**.
  - Evaluate all adjacent nodes and sort them according to the evaluation function.
  - Select the top ***W*** nodes and connect them to their parent nodes in the tree.
  - Repeat until the goal node ***G*** is removed from the frontier list so a path has been found.

Example



- Consider the left example of searching a graph from **S** to **G**.
- For simplicity, only the edge weights will be used for evaluation.
- A beam width of **W** = 2 will be used, meaning only the top two nodes are considered at every level.

Source: https://www.codecademy.com/resources/docs/ai/search-algorithms/beam-search

Example

- Initialize the tree and add **S** to the **Visited_list** [S], **Node_cost**[I(6)]
- **S** only has one adjacent node, so it is added to the tree:
  Visited_list [S, I], **Node_cost**[N(13), H(14), C(37)]
- Explore **I**. Nodes **N** & **H** are the top two nodes and are added to the tree.
  Visited_list [S, I, N, H], **Node_cost**[F(19), L(31), K(47)]
- Nodes **F** (adjacent to **H** in the graph) and **L** (adjacent to **N** in the graph) are next:
  Visited_list [S, I, N, H, F, L], **Node_cost**[M(30), C(40), G(41), L(54)]
- The incompleteness is evident here: Node **G** appears in the **Frontier_list** but unfortunately, it is eliminated by the beam width. Nodes **M** and **C** follow:
  Visited_list [S, I, N, H, F, L, M, C], **Node_cost**[G(46)]
- It is in this step that node **G** has been found:
  Visited_list [S, I, N, H, F, L, M, C, G], **Node_cost**[ ]

Assignment

- From the last example, generate a tree structure in each step.

Pros and Cons of Beam Search

- **Pros:**
  - **Efficiency**: Beam Search efficiently navigates large search spaces by limiting the number of nodes expanded.
  - **Flexibility**: The algorithm can be adjusted for different problems by modifying the beam width and heuristic function.
  - **Scalability**: Suitable for problems where the solution paths are vast and complex, as it does not require all nodes to be stored in memory.
- **Cons**
  - **Suboptimality**: Beam Search may not find the optimal solution, especially if the beam width is narrow.
  - **Heuristic Dependency**: Beam Search's effectiveness depends on the quality of the heuristic function; poor heuristics can lead to suboptimal results.

Applications of Beam Search

- It is commonly used in natural language processing, machine translation, speech recognition, and character recognition.
- An encoder is used to process the text from the source language and Beam search selects the most probable words (the beam width) in the target language.
- The evaluation function in this context is the conditional probability of a word in the target language.

Greedy Search vs Beam Search Decoding: Examples

- What's Decoding Method in NLP?
- What is Greedy Search Decoding?
- Drawbacks of Greedy Search Decoding Method
- What is Beam Search Decoding?
- Difference: Greedy Search & Beam Search Decoding Method

What's Decoding Method in NLP?

- In NLP and language models, "decoding" is the process of converting the output into a human-readable format of words or tokens.
- Language models generate a probability distribution for each word in their vocabulary at every step in a sequence. For a given input, the model provides probabilities that indicate the likelihood of each word being the next one.
- A method to "decode" this probability distribution into a sequence of words.

What is Greedy Search Decoding?

- The model selects the word (token) with the highest probability at each step and adds it to the sequence until a stop condition is met, such as an end token or maximum sequence length.
- Given the sentence start: "My favorite fruit is ___."
- The model predicts the next word based on its training data by selecting the one with the highest probability, considering the sentence's broader context.
- **1st Step:** After "My", the model might have high probabilities for "favorite", "mother", "dog", etc. It picks "favorite" as it has the highest probability.
- **2nd Step:** For "My favorite", the next word might be "fruit", "color", "thing", etc. Again, it chooses the word with the highest probability, which is "fruit".
- **3rd Step:** Now, "My favorite fruit", options might be "is", "are", "was", etc. So "is", has the highest probability.
- **4th Step:** After "My favorite fruit is", the options could be "apple", "banana", "grapes", and so forth. If "apple" has the highest probability, it chooses that.
- So, with Greedy Search, the completed sentence might be: "My favorite fruit is apple."

What is Beam Search Decoding?

- **Decoding method of Beam search** that keeps track of multiple potential sequences at each step (the 'beam width').
- At each step, it generates multiple new sequences by appending every possible next word to each of the sequences it's currently considering.
- It then selects the top 'W' most probable sequences from these new sequences, where 'W' is the beam width.
- Imagine you're trying to predict the next word in the sentence: "I like to eat ___."
  - Now, many potential words to fill in the blank like "cake", "apples", "pasta", etc.

Beam Search

- Let beam width W=2. That considers the top 2 sequences (combinations of words) based on their probabilities.
- **1st Step**: The model predicts the probabilities for the next word after "I".
  Let's say the highest probabilities are for "like" and "am". So, Beam Search keeps these two sequences:
  - "I like",
  - "I am"
- **2nd Step**: Now, for each sequence, the model predicts the next word:
  - "I like", it might predict "to" and "eating".
  - "I am", it might predict "happy" and "eating".
- This gives us the sequences:
  - "I like to"
  - "I like eating"
  - "I am happy"
  - "I am eating"
- From these, it picks the top 2 sequences based on their probabilities.
  - "I like to"
  - "I am happy"

Beam Search

- **2nd Step**: Now, for each sequence, the model predicts the next word:
  - "I like", it might predict "to" and "eating".
  - "I am", it might predict "happy" and "eating".

  This gives us the sequences:
  - "I like to"
  - "I like eating"
  - "I am happy"
  - "I am eating"

  From these, it picks the top 2 sequences based on their probabilities.
  - "I like to"
  - "I am happy"

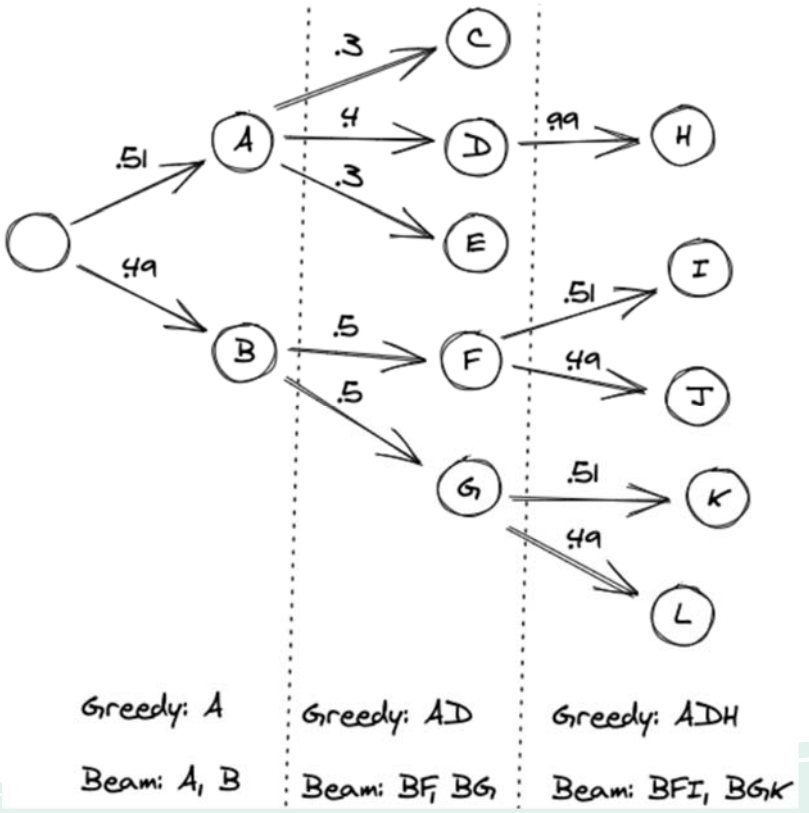
- **3rd Step**: Repeat the process. For "I like to", it predicts "eat" and "play". For "I am happy", it predicts "to" and "because".

  New sequences:
  - "I like to eat"
  - "I like to play"
  - "I am happy to"
  - "I am happy because"

  From these, the top 2 sequences might be:
  - "I like to eat"
  - "I am happy because"

Beam Search

- This process continues until an end-of-sequence token is encountered or until a set sequence length is reached.
- The picture on the right represents the difference in a visual manner.



Source: https://vitalflux.com/greedy-search-vs-beam-search-decoding-concepts-examples/

Algorithm

```
FUNCTION BEAM_SEARCH(Start_Node, Goal_State, Beam_Width w)
    # list of nodes in current beam
    CREATE List Frontier = [start]
    CREATE List Visited_Node = []
    WHILE Frontier is NOT empty
        IF Goal_State is in Frontier
            RETURN SUCCESS and extract_path(Goal_State)
        END IF
        CREATE List New_frontier = []
        # expand all nodes in the current frontier
        FOR Node in Frontier
            Add Node in Visited_Node
            Children = EXPAND(Node)
            # generate successors
            FOR Child in Children
                IF Child in Visited_Node
                    Add Child to New_frontier
                END IF
            END FOR
        END FOR
        IF New_frontier is empty
            RETURN FAILURE
        END IF
        # sort by heuristic or evaluation function f(n)
        New_frontier = SORT_BY_COST(New_frontier)
        # keep only the best w nodes
        Frontier = FIRST_w(New_frontier, w)
    END WHILE
    RETURN FAILURE
END FUNCTION
```


Difference: Greedy & Beam Search Decoding Method

Criteria	Greedy Search	Beam Search
Decoding method	Picks the next word (token) with the highest immediate probability.	Track multiple potential sequences by selecting the top 'W' sequences at each step based on their cumulative probabilities.
When to use	When computational efficiency is prioritized, and you need a quick, decent solution.	To balance computational cost and output quality is required, especially beneficial when higher quality or varied output is desired.
Flexibility	Relatively rigid as it always selects the word with the highest immediate probability.	More flexible keeping multiple sequences in consideration, leading to a better exploration of possible outputs.
Risk of Local Minima	High. Often get stuck in local optima because it doesn't consider multiple possible sequences.	Reduced. Tracking multiple sequences helps avoid local optima and find better outcomes.
Quality of Output	Can be suboptimal, especially in cases where looking ahead is crucial for a better sequence.	Produces better and more diverse results than greedy search, especially when the beam width is appropriately set.
Memory	Minimal. Only needs to remember the current sequence.	Higher. Has to remember multiple sequences (beam width) at each step.

AI